

Web Programming and Python: Assignment-11

Name: Suraj Shah

Admission Number: I24AI012

1. Write a Pandas program to create
 - a) Date time object for Jan 15 2012.
 - b) Specific date and time of 9:20 pm.
 - c) Local date and time.
 - d) A date without time.
 - e) Current date.
 - f) Time from a date time.
 - g) Current local time.

Code:

```
import pandas as pd
from datetime import datetime

dt_a=pd.Timestamp('2012-01-15')
print("Date time object for Jan 15 2012:",dt_a)

dt_b=pd.Timestamp('2025-04-09 21:20')
print("Specific date and time of 9:20 pm:",dt_b)

dt_c=pd.Timestamp.now()
print("Local date and time:",dt_c)

dt_d=pd.to_datetime('2025-04-09').date()
print("A date without time:",dt_d)

dt_e=pd.Timestamp.today().date()
print("Current date:",dt_e)

dt_f=pd.Timestamp.now().time()
print("Time from a date time:",dt_f)

dt_g=datetime.now().time()
print("Current local time:",dt_g)
```

Output:

```
Date time object for Jan 15 2012: 2012-01-15 00:00:00
Specific date and time of 9:20 pm: 2025-04-09 21:20:00
Local date and time: 2025-04-09 19:34:27.049311
A date without time: 2025-04-09
Current date: 2025-04-09
Time from a date time: 19:34:27.051511
Current local time: 19:34:27.051648
```

2. Write a Pandas program to convert all the string values to upper, lower cases in a given pandas series. Also find the length of the string values.

```
s = pd.Series(['X', 'Y', 'T', 'Aaba', 'Baca', 'CABA', None, 'bird', 'horse', 'dog'])
```

Code:

```
import pandas as pd

s = pd.Series(['X', 'Y', 'T', 'Aaba', 'Baca', 'CABA', None, 'bird', 'horse', 'dog'])

upper_case=s.str.upper()
print("Upper case:\n",upper_case)

lower_case=s.str.lower()
print("Lower case:\n",lower_case)

string_length=s.str.len()
print("Length of each string:\n",string_length)
```

Output:

```
● Upper case:
 0      X
 1      Y
 2      T
 3    AABA
 4    BACA
 5    CABA
 6    None
 7    BIRD
 8    HORSE
 9     DOG
dtype: object
Lower case:
 0      x
 1      y
 2      t
 3    aaba
 4    baca
 5    caba
 6    None
 7    bird
 8    horse
 9     dog
dtype: object
Length of each string:
 0     1.0
 1     1.0
 2     1.0
 3     4.0
 4     4.0
 5     4.0
 6    NaN
 7     4.0
 8     5.0
 9     3.0
dtype: float64
```

3. After accidentally leaving an ice chest of fish and shrimp in your car for a week while you were on vacation, you're now in the market for a new vehicle. Your insurance didn't cover the loss, so you want to make sure you get a good deal on your new car.

Given a Series of car asking_prices and another Series of car fair_prices, determine which cars for sale are a good deal. In other words, identify cars whose asking price is less than their fair price.

The result should be a list of integer indices corresponding to the good deals in asking_prices.

Code:

```
import pandas as pd

asking_prices=pd.Series([13000, 15000, 18000, 12000, 16000])
fair_prices=pd.Series([14000, 15500, 17500, 12500, 15000])

good_deals=asking_prices<fair_prices

good_deals_indices=good_deals[good_deals].index.tolist()

print("Indices of good deals:", good_deals_indices)
```

Output:

```
● Indices of good deals: [0, 1, 3]
```

4. Whenever your friends John and Judy visit you together, y'all have a party. Given a DataFrame with 10 rows representing the next 10 days of your schedule and whether John and Judy are scheduled to make an appearance, insert a new column called `days_til_party` that indicates how many days until the next party. `days_til_party` should be 0 on days when a party occurs, 1 on days when a party doesn't occur but will occur the next day, etc.

Code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'John': [True, False, True, False, True, False, False, True, True, False],
    'Judy': [True, True, False, False, True, False, True, True, False, False]
})

party_days = (df['John'] & df['Judy']).values

days_til_party = np.full(len(df), fill_value=np.nan)

next_party = None
for i in reversed(range(len(df))):
    if party_days[i]:
        next_party = i
        days_til_party[i] = 0
    elif next_party is not None:
        days_til_party[i] = next_party - i

df['days_til_party'] = days_til_party.astype(int)

print(df)
```

Output:

```
df['days_til_party'] = days_til_party.astype(int)
   John  Judy  days_til_party
0  True  True              0
1  False True              3
2  True  False             2
3  False False             1
4  True  True              0
5  False False             2
6  False True              1
7  True  True              0
8  True  False -9223372036854775808
9  False False -9223372036854775808
```

5. Given a dataset of concerts, count the number of concerts per (artist, venue), per year month. Make the resulting table be a wide table - one row per year month with a column for each unique (artist, venue) pair. Use the cross product of the artists and venues Series to determine which (artist, venue) pairs to include in the result.

Code:

```
import pandas as pd

df = pd.DataFrame({
    'artist': ['A', 'B', 'A', 'A', 'C', 'B', 'A'],
    'venue': ['X', 'Y', 'X', 'Y', 'Z', 'X', 'Z'],
    'date': pd.to_datetime([
        '2024-01-10', '2024-01-15', '2024-01-20',
        '2024-02-05', '2024-02-20', '2024-02-25', '2024-03-01'
    ])
})

df['year_month'] = df['date'].dt.to_period('M')

artists = df[['artist']].drop_duplicates()
venues = df[['venue']].drop_duplicates()

# Step 3: Cross join using pandas merge
cross = artists.merge(venues, how='cross')

grouped = (
    df.groupby(['year_month', 'artist', 'venue'])
      .size()
      .rename('count')
      .reset_index()
)

all_months = df[['year_month']].drop_duplicates()

full_index = all_months.merge(cross, how='cross')

full_data = (
    full_index.merge(grouped, on=['year_month', 'artist', 'venue'],
                    how='left')
    .fillna(0)
)

pivot = full_data.pivot_table(
    index='year_month',
    columns=['artist', 'venue'],
```

```
        values='count',
        fill_value=0
    )

pivot = pivot.reset_index()

print(pivot)
```

Output:

	artist	year_month	A			B			C		
			X	Y	Z	X	Y	Z	X	Y	Z
0		2024-01	2.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
1		2024-02	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0
2		2024-03	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0